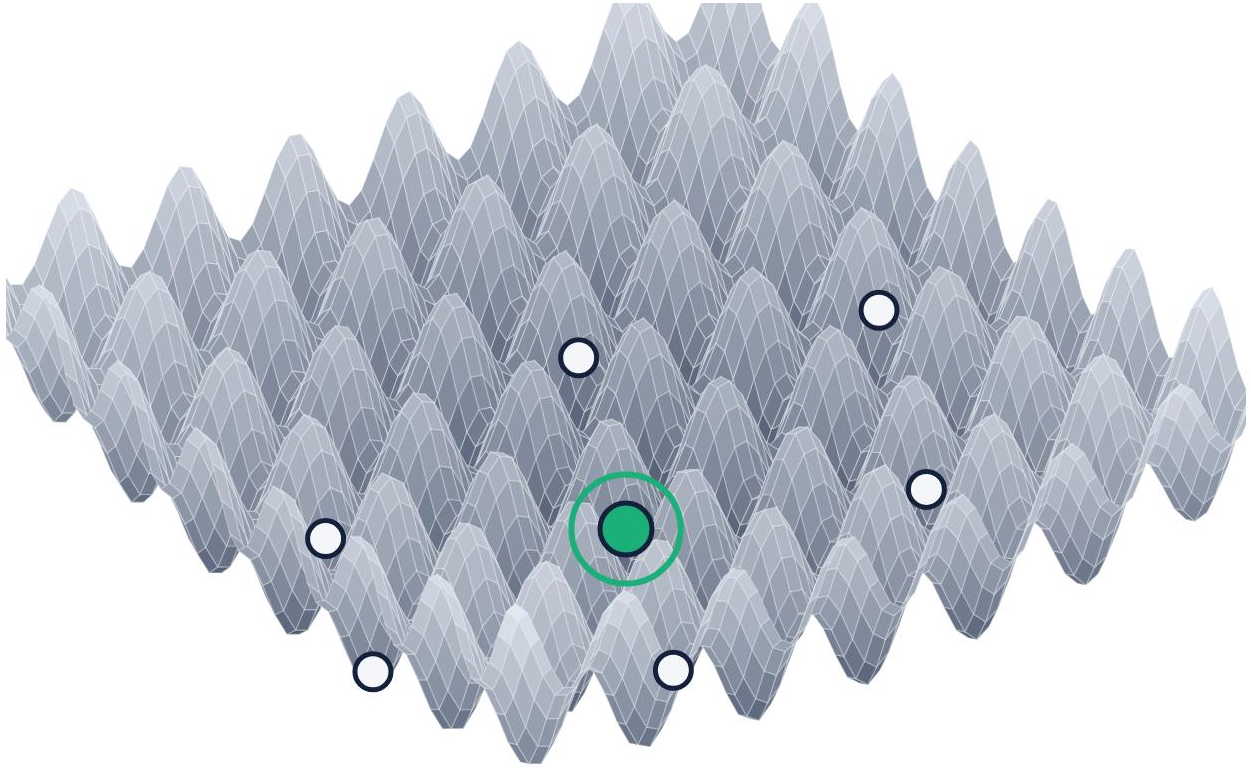
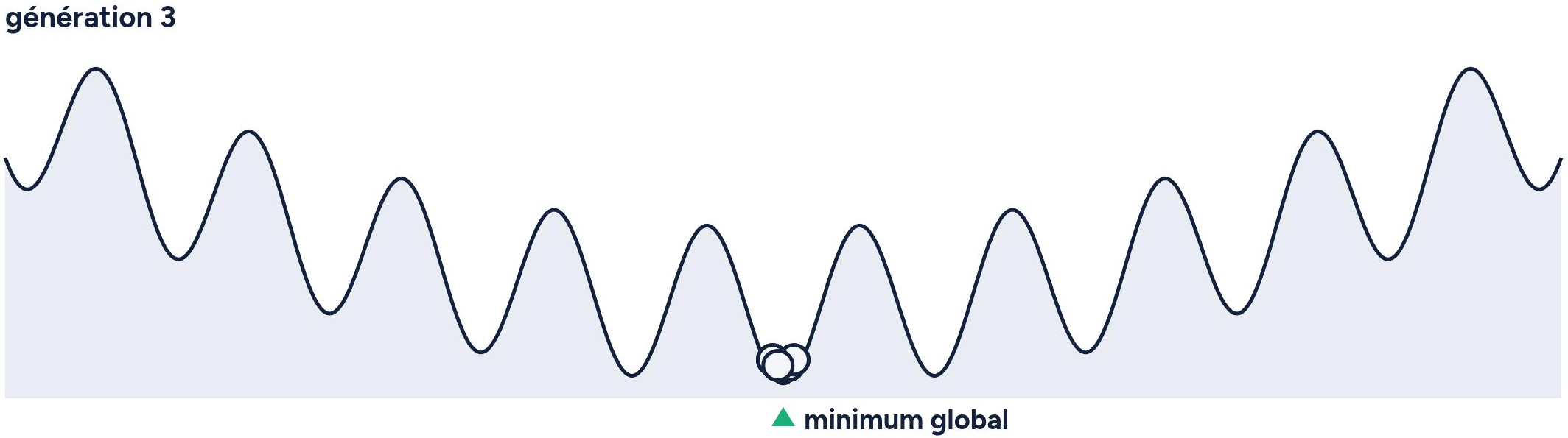
Évolution différentielle sur GPU



Du PSO fourni à trois versions CUDA du DE, comparées à la version séquentielle.

Zainab Elamri, Omar Elsemry, Mohamed De Franceschi
Master IM, UHA. Calcul parallèle, L. Idoumghar

Beaucoup de puits, un seul minimum



Une recherche locale descend dans le puits le plus proche et y reste.

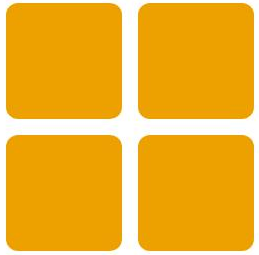
Une population explore tout le terrain et finit dans le bon puits.

4 fonctions : Sphere, Rastrigin, Rosenbrock, Griewank

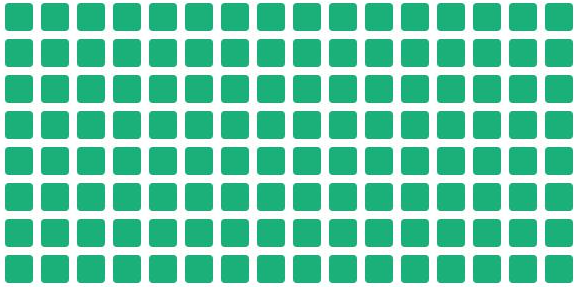
D = 10, 50, 100 et Pop = 50, 100, 500

Budget : $10^4 \times D$ évaluations

CUDA en une minute



CPU : peu de cœurs rapides



GPU : des milliers de petits cœurs

`kernel<<<blocs, threads>>>()`

chaque thread calcule son numéro :

`i = blockIdx.x * blockDim.x + threadIdx.x`

- Mémoire globale : visible par tous les threads, mais lente
- Mémoire partagée : rapide, mais seulement dans un bloc
- Copier entre CPU et GPU coûte cher
- Lancer un kernel coûte environ 3 µs
- Deux kernels à la suite : le 2e attend que le 1er ait fini pour tous les threads

Le PSO : un essaim de particules

Chaque particule a une position **x** et une vitesse **v**.

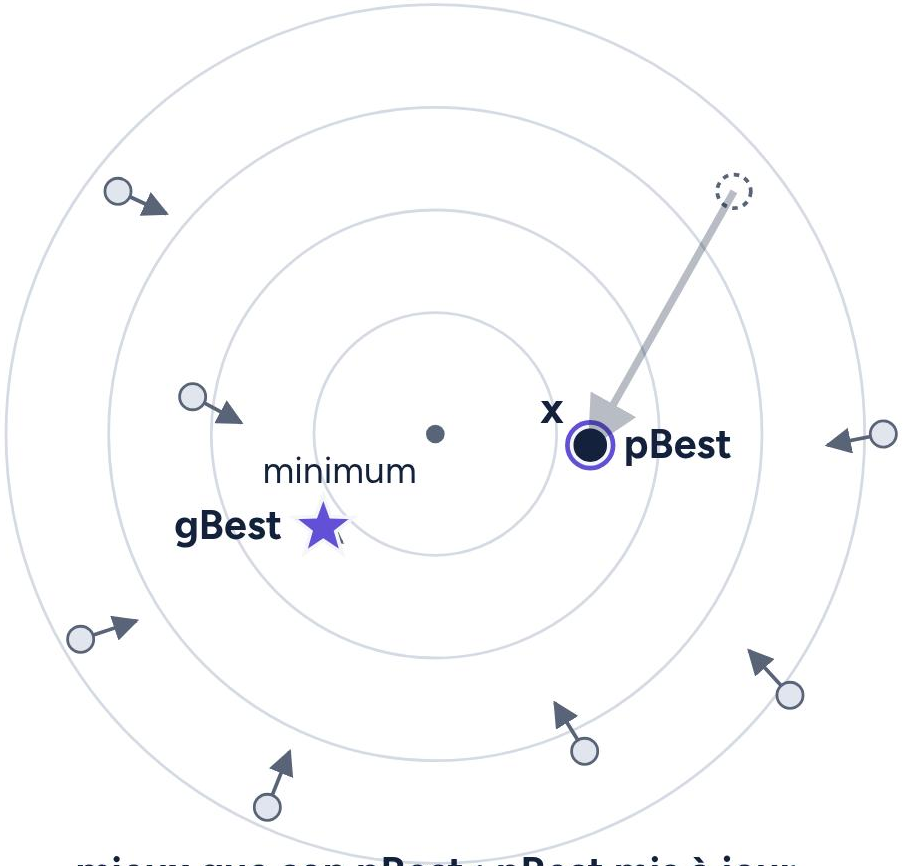
pBest : la meilleure position que cette particule a déjà vue.

gBest : la meilleure position de tout l'essaim.

Nouvelle vitesse = un peu de l'ancienne + attirance vers pBest + attirance vers gBest.

La particule avance. Si c'est mieux, son pBest est mis à jour.

Dans le code fourni : w = 0,5 et $c_1 = c_2 = 1,5$



mieux que son pBest : pBest mis à jour

Le PSO fourni, côté CUDA



Le gBest repart sur le GPU et on recommence. 2 copies et 1 boucle CPU à chaque itération : c'est ce qui le rend lent.

Le DE en trois étapes

Pour chaque individu $\mathbf{x_i}$ de la population :

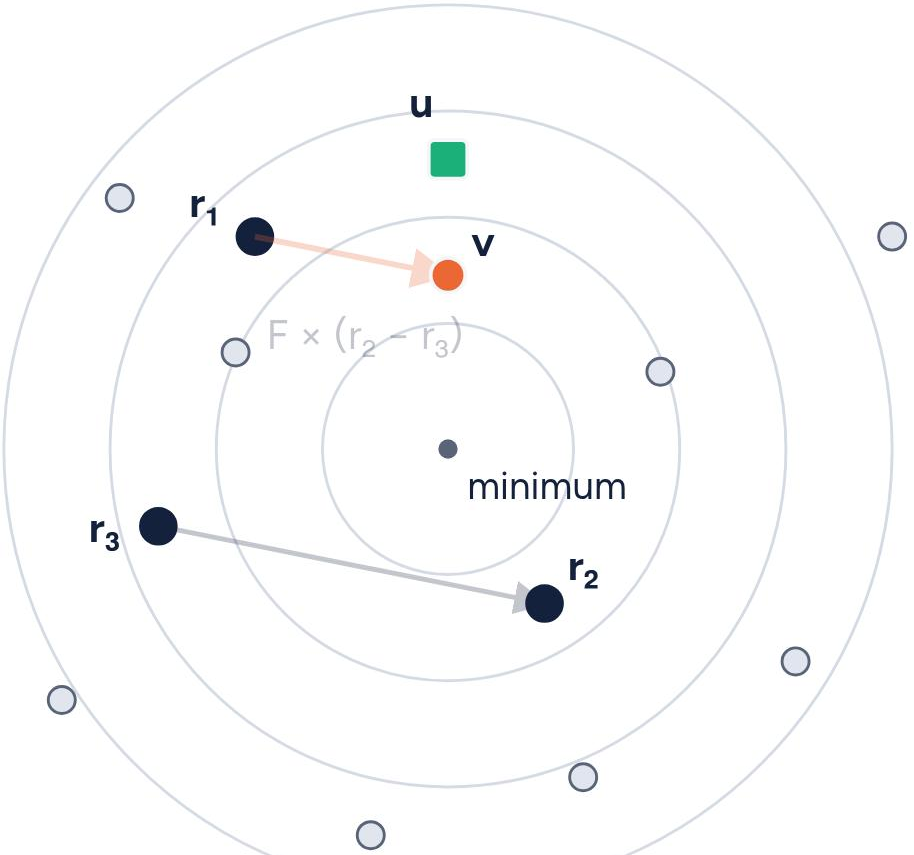
On tire trois autres individus r_1, r_2, r_3.

Mutation : $v = x_{r1} + F\,(x_{r2} - x_{r3})$

Croisement : l'essai u prend certaines coordonnées de v, les autres de x_i

Sélection : si $f(u) \leq f(x_i)$, u remplace x_i

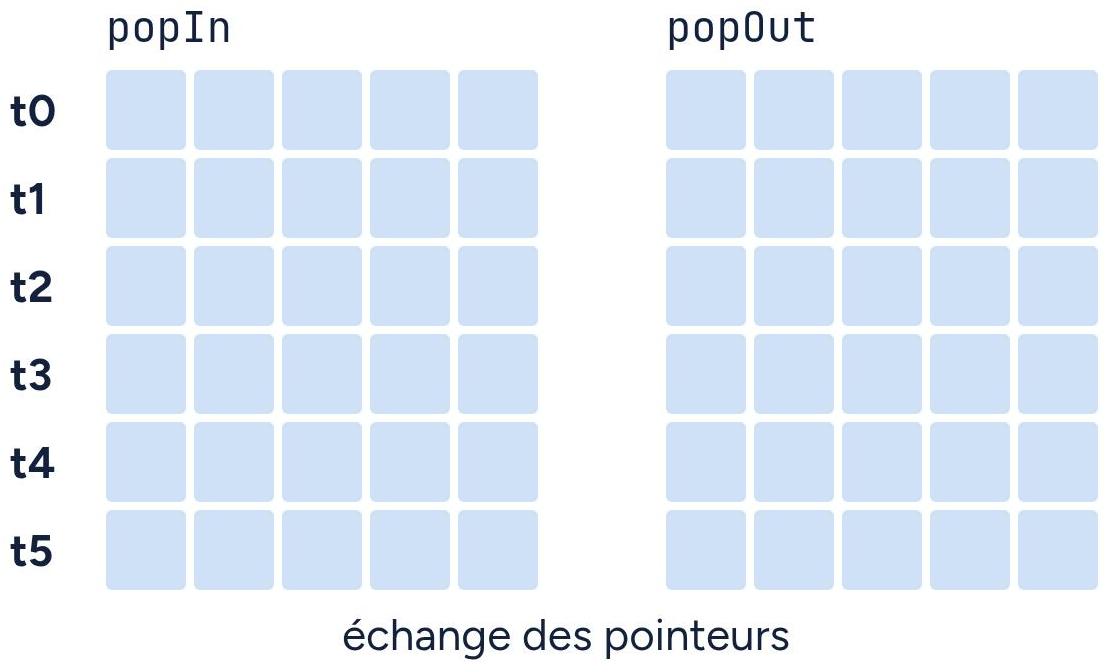
F = 0,5 et CR = 0,9 (probabilité de prendre la coordonnée du mutant)



f(u) < f(x_i) : u remplace x_i

v1 : un seul kernel

- 1 thread = 1 individu
- Chaque thread fait tout : mutation, croisement, évaluation, sélection, en bouclant sur les D coordonnées
- On lit dans `popIn`, on écrit dans `popOut`, puis on échange les deux
- 1 lancement de kernel par génération, une seule copie au début et une à la fin



`kernelDEGeneration`

mutation → croisement → évaluation → sélection

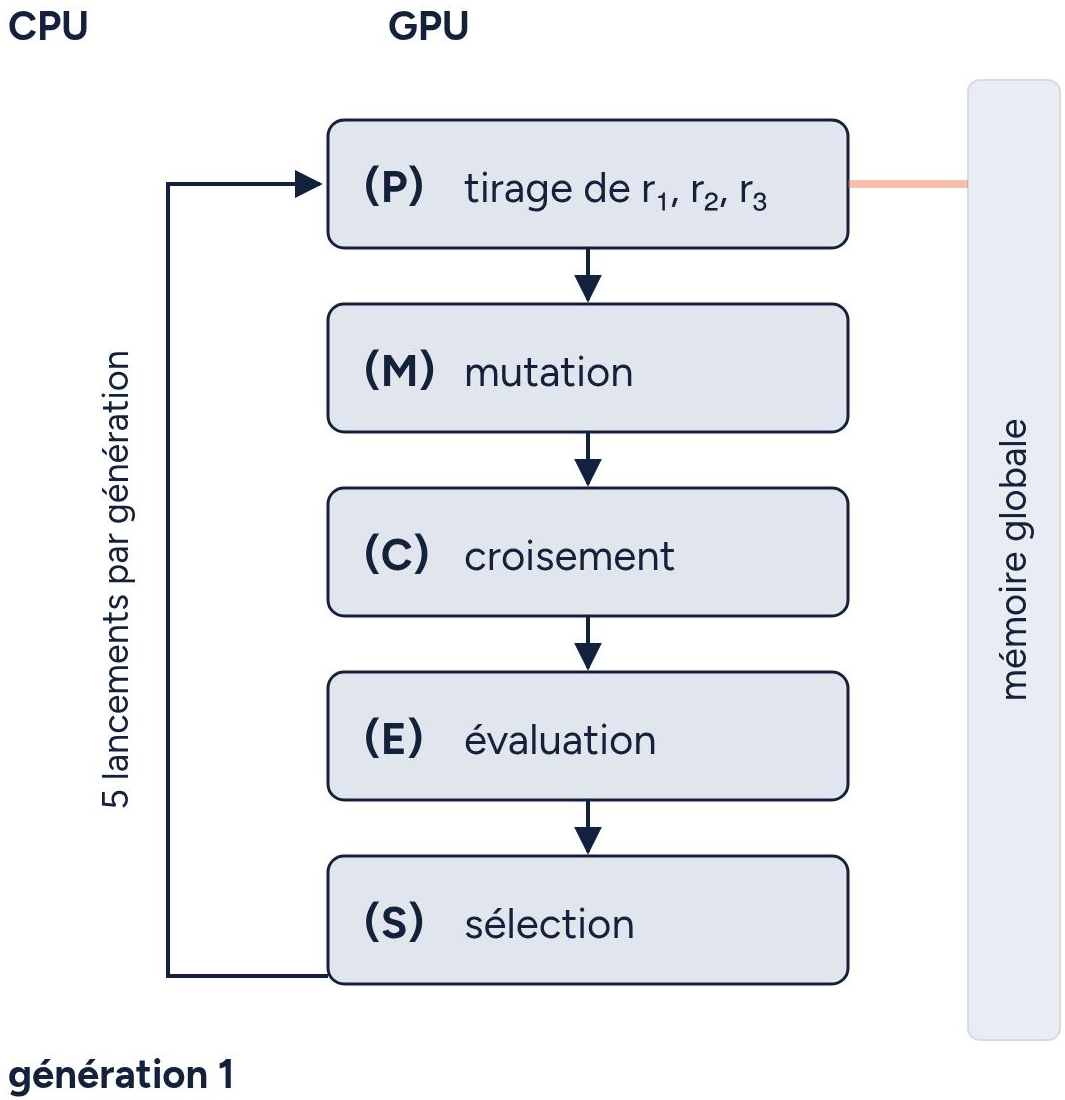
v2 : un kernel par étape

Toujours 1 thread = 1 individu.

Les kernels se passent leurs résultats par la mémoire globale.

Plus besoin de double buffer : quand (S) démarre, (M) a fini partout.

Mêmes résultats que la v1, au chiffre près, sur les 360 runs.



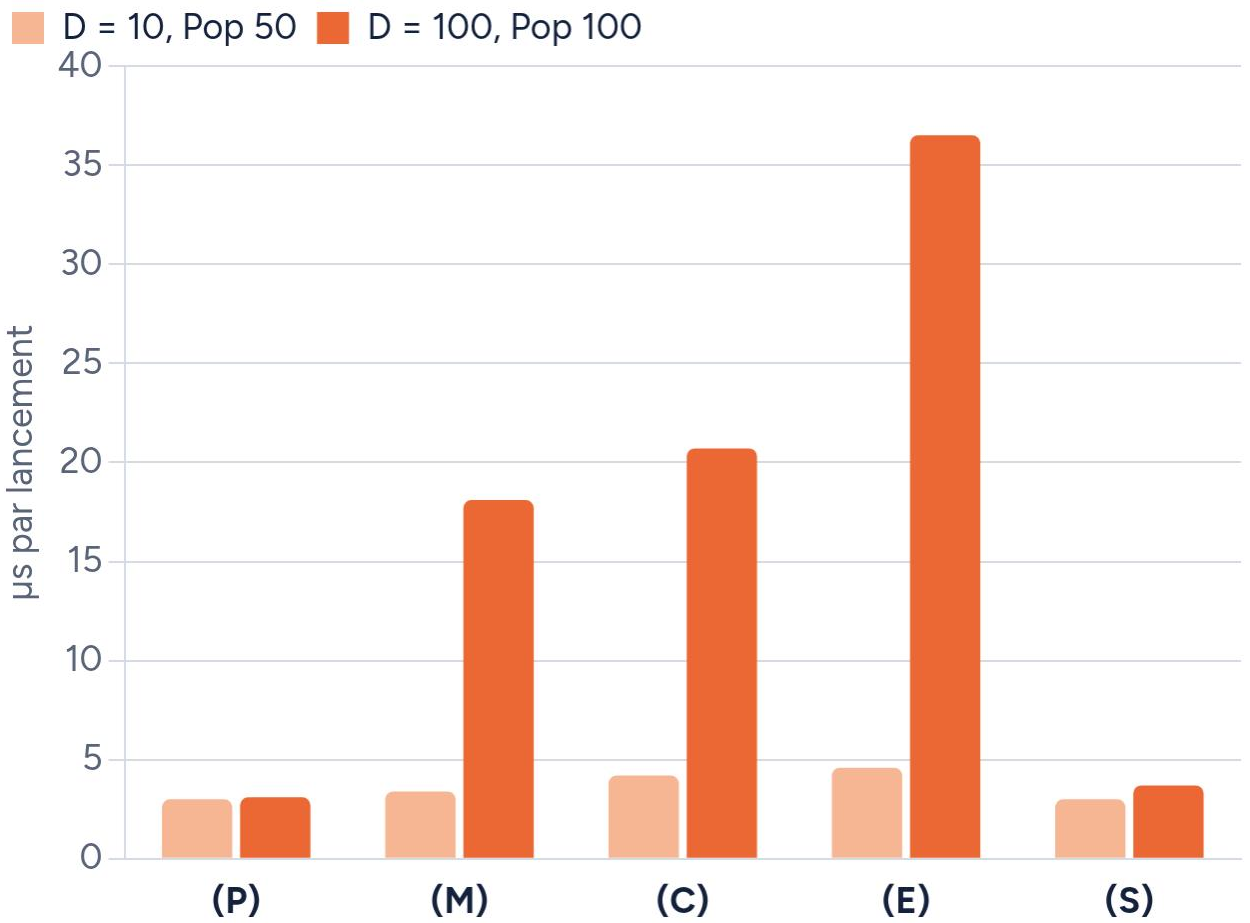
Pourquoi la v2 est plus lente

1,4× plus lente que la v1 (médiane)

D = 10 : 3 à 5 µs par kernel, surtout le coût du lancement. Et 5 lancements par génération au lieu d'un.

D = 100 : M, C et E bouclent sur 100 coordonnées en mémoire globale.

Temps d'un lancement de chaque kernel de la v2 (µs), Rastrigin



v3 : un thread par coordonnée

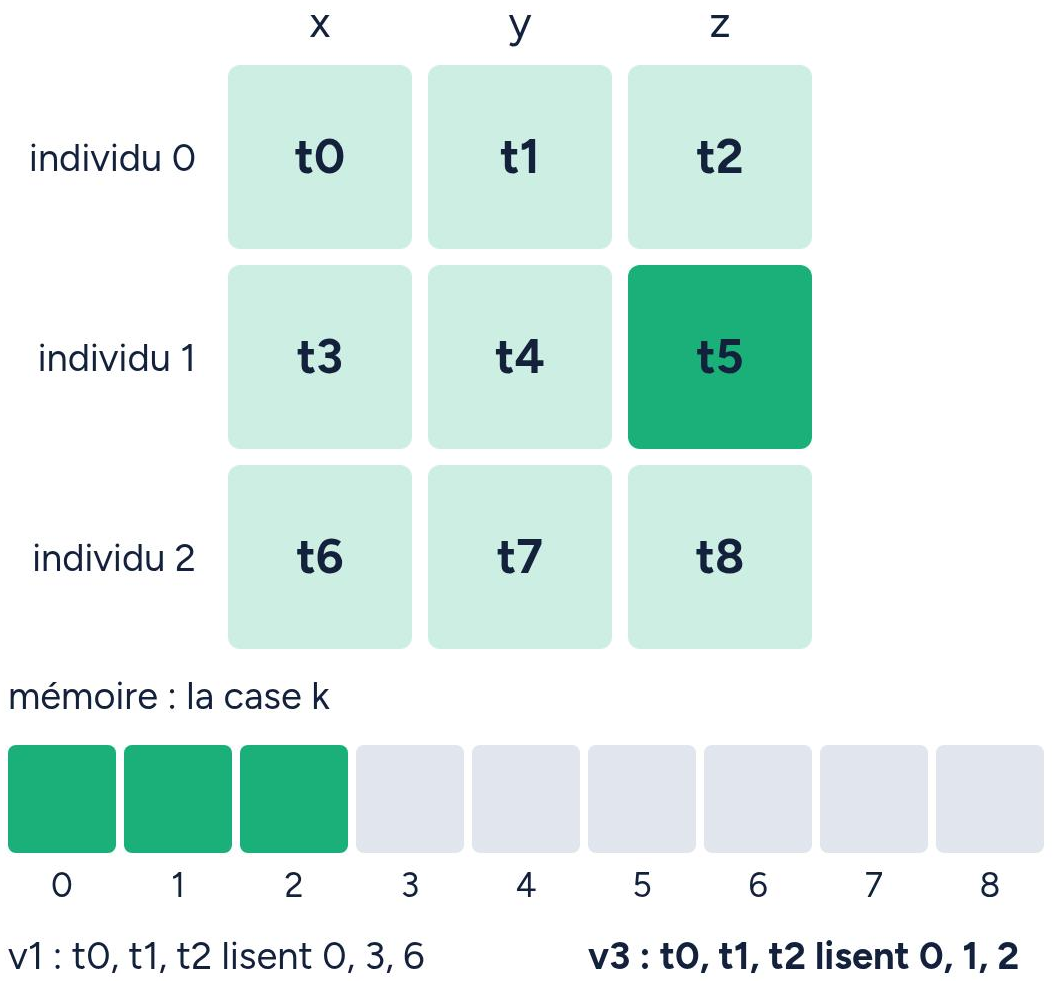
3 individus, 3 dimensions (x, y, z)

9 threads au lieu de 3.

Pop 100 et D = 100 : 10 000 threads

Le thread k traite :
`i = k / D` (l'individu)
`d = k % D` (la coordonnée)

Des threads voisins lisent des cases voisines : le GPU groupe les lectures.



v3 : les 5 kernels, thread par thread



E : f(u) est une somme sur les coordonnées. Chaque thread calcule le terme de sa coordonnée, puis on additionne en mémoire partagée.

une case = le terme d'une coordonnée de u, calculé par son thread (Rastrigin, D = 10)



16 cases (puissance de 2) : les 6 en trop valent 0

La case 0 contient la somme : 9,6

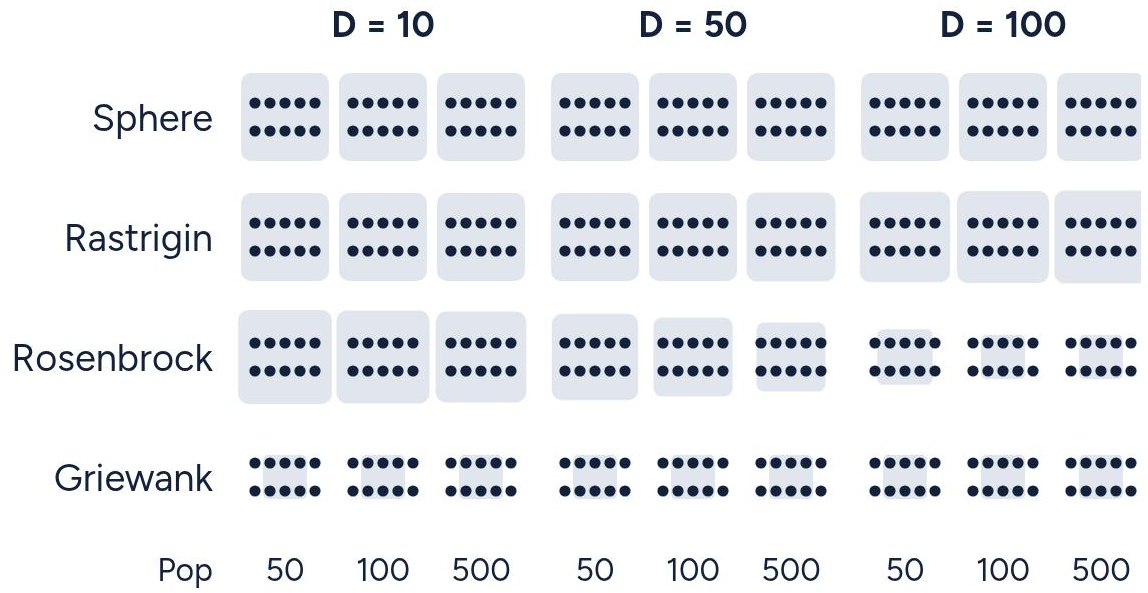
__syncthreads() entre deux tours

f(u) = 9,6 − 330 = −320,4

Protocole

- 4 fonctions × 3 dimensions × 3 populations = 36 cas
- 10 runs par cas, même graine : tous les programmes partent des mêmes populations
- Temps réel de l'appel, copies comprises
- Tests de Wilcoxon et Kruskal-Wallis, seuil 5 %

RTX 2080 Ti, i7-8700K, CUDA 13.4

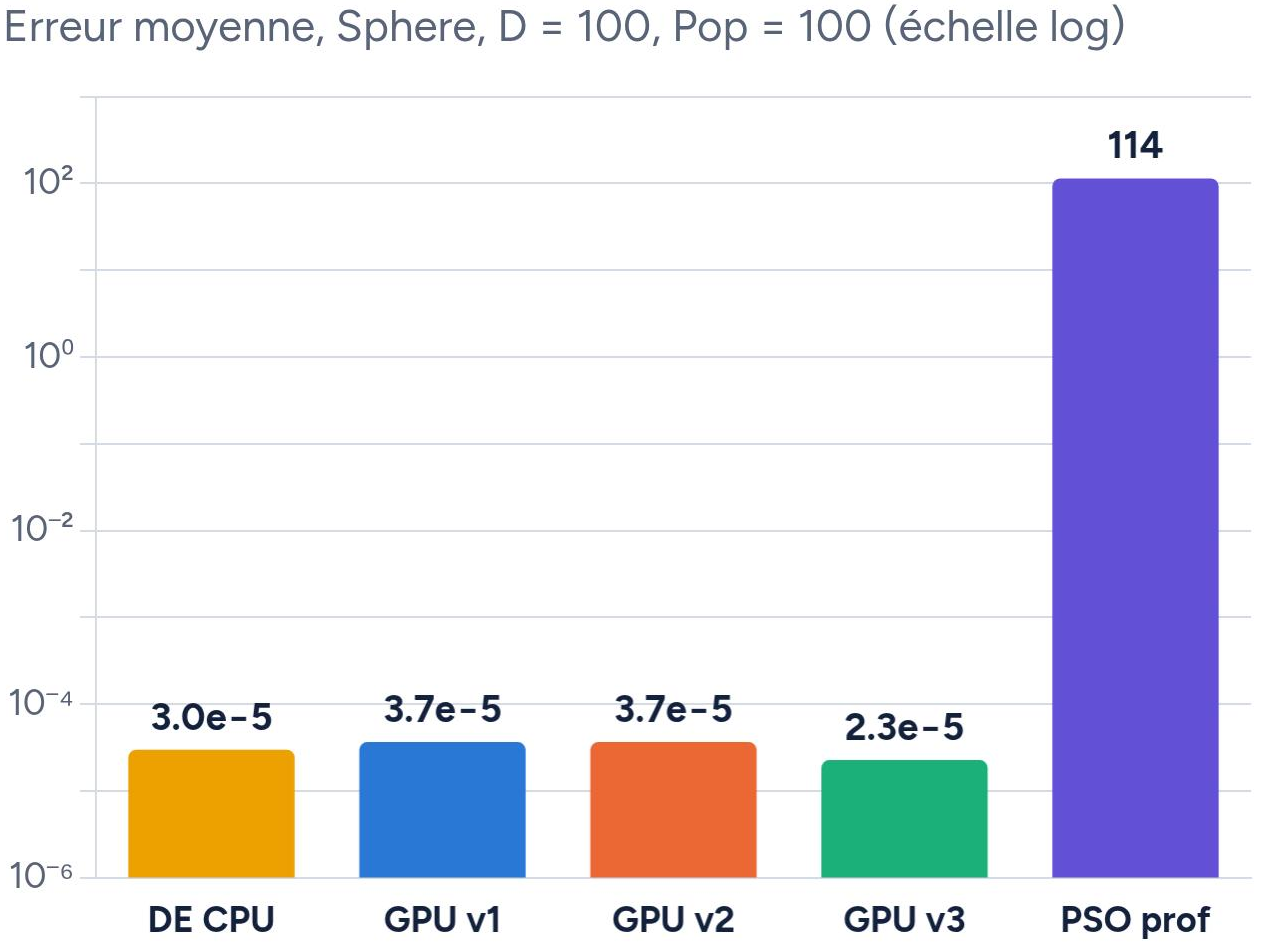


Une case = un cas (fonction, D, Pop). Un point = un run.

1800 runs (5 programmes) + 720 avec CR = 0,3

Même qualité pour les 4 versions du DE

- Différence significative CPU / v1 : 2 cas sur 36, CPU / v3 : 0 sur 36
- Avec 5 %, on attend environ 2 cas sur 36 par hasard
- Le DE bat le PSO dans 34 cas sur 36

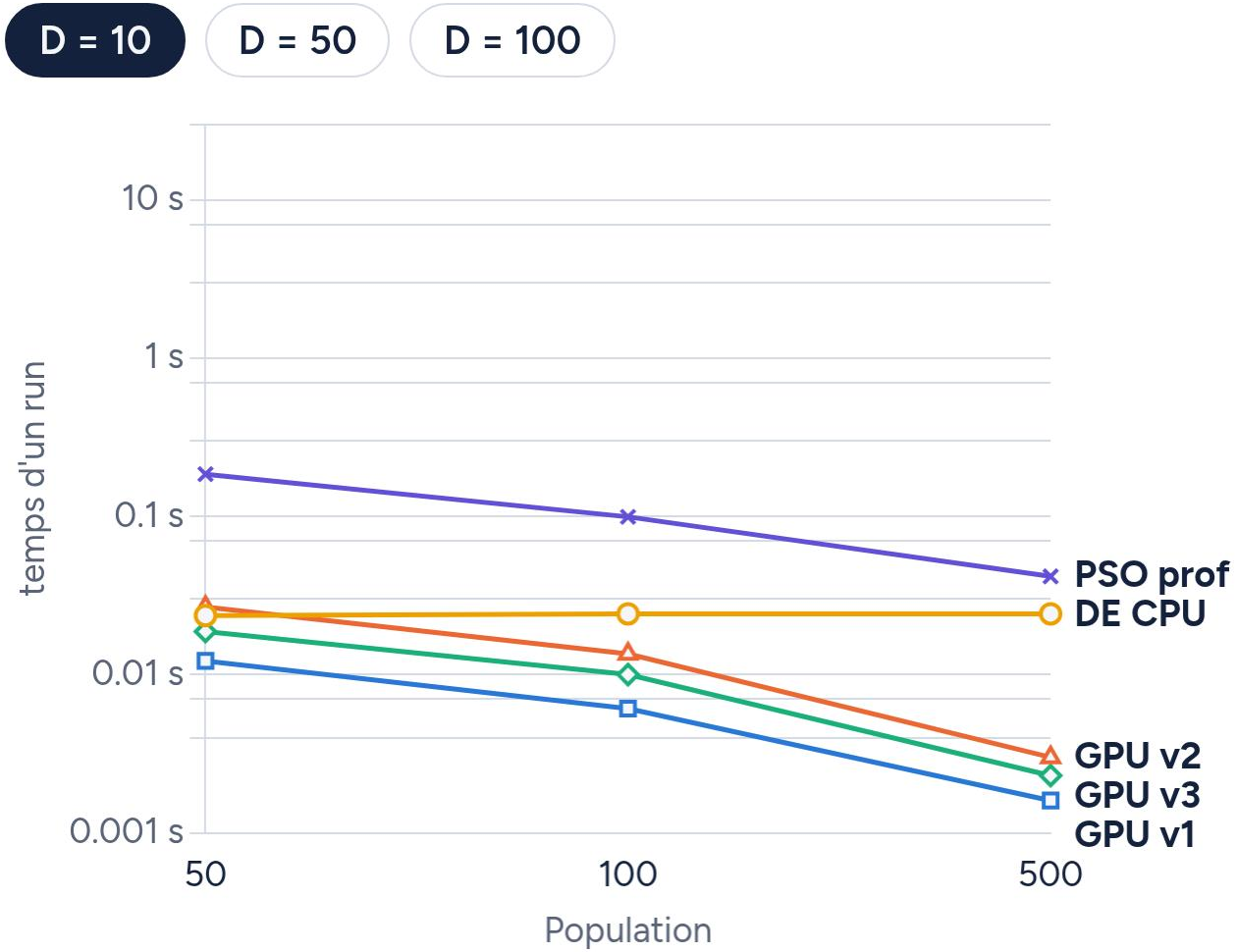


Temps de calcul

89× v3 contre le CPU (max)

Médianes sur les 36 cas :
v1 3,5×, v2 2,3×, v3 8,4×

D = 10 : la v3 est environ 1,5× plus lente que la v1. Trop peu de travail par kernel, et 5 lancements par génération.



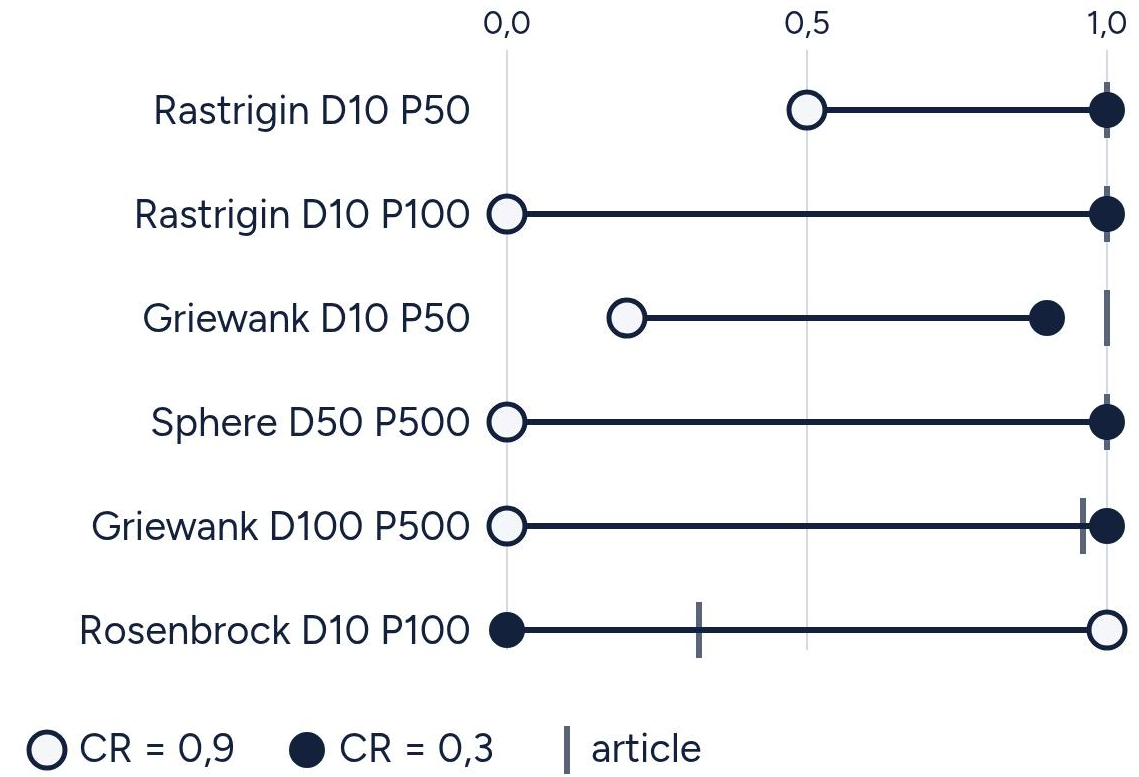
Article et essai avec CR = 0,3

Sphere et Griewank, D ≥ 50, Pop 50 et 100 : taux de succès aussi bon que l'article, ou meilleur.

CR = 0,3 (valeur de l'article) : mieux dans 19 cas, moins bien dans 17.

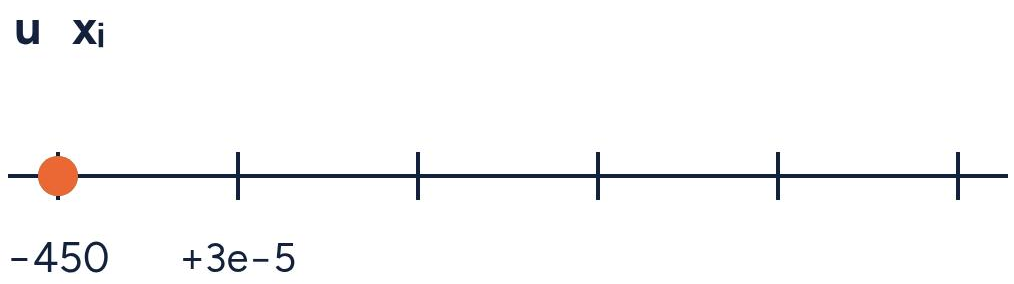
Article : succès si erreur < 10^{-8}. Nous : < 10^{-4}, plus facile (voir limites).

Taux de succès du DE GPU sur 10 runs



Limites

les valeurs qu'un float peut stocker

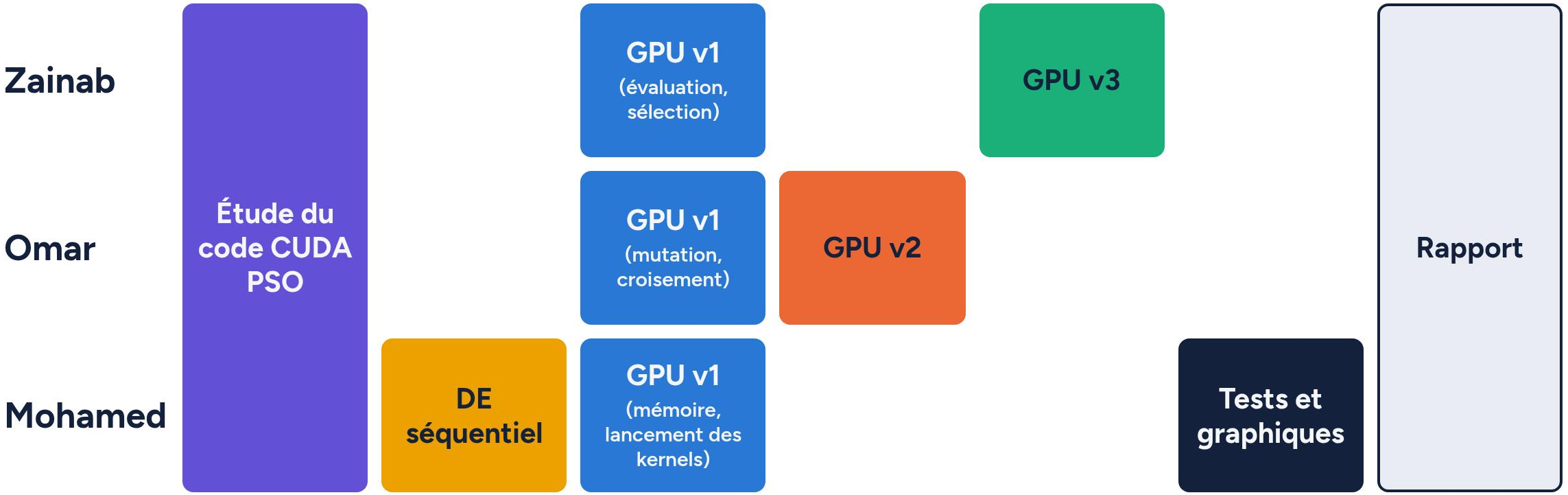


même valeur : la sélection ne voit plus la différence

Près de 450, deux `float` voisins sont séparés d'environ 3×10^{-5}.

- f est calculée en `float` avec la constante (−450…) dedans : l'erreur bloque vers 10^{-5}
- Pas de `double` : environ 32× plus lent sur notre carte. Mieux : calculer f sans la constante
- PSO fourni gardé tel quel (seul Griewank corrigé), une seule autre valeur de CR testée

Qui a fait quoi



De gauche à droite : l'ordre dans lequel le projet a avancé.

Conclusion

- **v1** un seul kernel : jusqu'à 27× plus rapide que le CPU
- **v2** un kernel par étape : plus clair, mais plus lent (5 lancements par génération)
- **v3** un thread par coordonnée : la plus rapide dès $D \geq 50$, jusqu'à 89×
- Même qualité pour toutes les versions du DE, bien meilleure que le PSO fourni
- Suite : regrouper M, C, E et S en un kernel en gardant un thread par coordonnée, comme cudaDEi

Merci

Questions ?

Zainab Elamri, Omar Elsemry, Mohamed De Franceschi

